Studio Nx

AI 구현 설명 09 — 프론트 공통 JS 뼈대

2026-10-02

AI 구현 설명 09 — 프론트 공통 JS 뼈대

항목	내용
작성	Claude (AI)
작성일	2026-10-02
작업	P1-02 프론트 공통 JS: API 호출 함수, 401이면 재발급, 로그인 확인 (**뼈대**). 이전 문서: 08_backend_core.md
브랜치	`feature/phase1/p1-02-frontend-api-core` (P1-03 브랜치 위에서 시작)
원칙	JS는 **템플릿만** — 함수 이름, 인자, 처리 순서(TODO). 실제 구현은 사용자

---

1. 무엇을 만들었나

파일	상태	내용
`static/js/core/api.js`	뼈대	`apiFetch`, `api.get/post/put/patch/delete`, `refreshAccessToken`, 토큰 저장 함수
`static/js/core/auth.js`	뼈대	`isLoggedIn`, `requireLogin`, `loginUrl`, `parseLoginHash`, `safeNextPath`, `logout`
`static/js/core/auth-ui.js`	주석 수정	`initAuthState`가 `auth.js`의 `isLoggedIn`을 쓰도록 안내
`static/js/pages/home.js`	주석 수정	로그인 모달 열기·전송이 `parseLoginHash`, `api.post`를 쓰도록 안내

완성되어 있는 것은 `ApiError` 클래스, 토큰 저장 함수 3개(`get/set/clearAccessToken`), `api.get` 같은 짧은 함수들입니다. 나머지는 TODO입니다.

2. 왜 이렇게 했나

2.1 토큰을 어디에 두나

토큰	위치	이유
Access Token (30분)	`api.js` 모듈 안 변수 (메모리)	`localStorage`에 두면 악성 스크립트(XSS)가 읽어 갈 수 있음
Refresh Token (세션 또는 3일)	HttpOnly 쿠키 (서버가 설정)	JS에서 아예 보이지 않음. `/auth/refresh` 요청에 브라우저가 자동으로 실음

페이지를 옮기면 메모리가 비워집니다. 그래서 첫 요청 전에 `refreshAccessToken()`으로 다시 받습니다. 자동 로그인(3일)을 체크했다면 브라우저를 다시 열어도 쿠키가 남아 있어 재발급이 됩니다.

2.2 요청 흐름 (`apiFetch`)

```
api.get('/users/me')
 → 토큰 없으면 refreshAccessToken()
 → Authorization: Bearer <token> 붙여 요청
 → 401 + code UNAUTHORIZED ?
      예 → refreshAccessToken() 성공 → 같은 요청 한 번 더
```

```
          실패 → ApiError 던짐 (화면은 requireLogin 흐름)
→ 그 외 실패 → 서버의 {"error": {code, message, details}} 로 ApiError
→ 성공 → JSON (204면 null)
```

- **재발급은 한 번만:** 여러 요청이 동시에 401을 받아도 `refreshing`(진행 중인 Promise)을 같이 기다리게 해서 `/auth/refresh`는 한 번만 보냅니다.
- **오류 모양:** P1-03에서 정한 서버 오류 형식을 그대로 `ApiError`의 `code`·`message`·`details`로 옮깁니다. 화면 코드는 `err.code`로 분기하면 됩니다(예: `EMAIL_TAKEN`).

2.3 로그인 확인 (`auth.js`)

로그인 페이지가 없으므로(PRD 01 FR-105), 로그인이 필요한 페이지는 맨 위에서 `requireLogin()`을 부릅니다. 로그인이 안 됐으면 `home.html#login?next=<원래 경로>`로 보냅니다.

함수	하는 일
`loginUrl(next)`	`home.html#login?next=%2FmyPage.html` 같은 주소 만들기
`parseLoginHash()`	메인 페이지에서 주소를 읽어 `{ open, next }` 반환 → 모달 열기
`safeNextPath(next)`	`next`가 같은 사이트 경로인지 검사. 외부 주소(`//evil.com`, `javascript:`)는 거부

`safeNextPath`가 필요한 이유는 **open redirect** 공격입니다. 누군가 `home.html#login?next=https://가짜사이트`라는 링크를 보내면, 로그인한 사람이 가짜 사이트로 이동할 수 있습니다.

3. 검증 결과 (2026-10-02)

확인	결과
문법 검사 (`node --check`)	5개 파일 모두 통과
브라우저 로드 (`home.html`, `join.html`)	콘솔 오류 없음. `api.js`, `auth.js` import 정상
뼈대 동작	`api.get()`을 부르면 `ApiError(NOT_IMPLEMENTED)`를 던짐 — 구현 전임을 바로 알 수 있게

4. 남은 일 (사용자)

할 일	파일	선행
`apiFetch`, `refreshAccessToken` 구현	`core/api.js`	`/auth/refresh` API (P1-05)
`isLoggedIn`, `requireLogin`, `loginUrl`, `parseLoginHash`, `safeNextPath`, `logout`	`core/auth.js`	— (`safeNextPath`, `loginUrl`, `parseLoginHash`는 바로 구현 가능)
로그인 모달·가입 화면 동작	`pages/home.js`, `pages/join.js`, `core/auth-ui.js`	P1-07

5. 자주 받을 질문 대비

- **Q. Access Token을 `sessionStorage`에 두면 안 되나?** 페이지를 옮겨도 남아 편하지만, XSS에 노출되는 건 `localStorage`와 같습니다. 이 서비스는 리프레시 쿠키로 언제든 다시 받을 수 있으므로 메모리에만 둡니

다.

- **Q. 401이면 무조건 재발급하나?** `code`가 `UNAUTHORIZED`일 때만입니다. 로그인 실패(`INVALID_CREDENTIALS`)도 401이지만 재발급 대상이 아닙니다.
- **Q. 왜 P1-03 브랜치 위에서 시작했나?** 두 작업이 같은 문서 파일(`build.js`, 맥락 문서)을 고치기 때문입니다. P1-03 PR이 먼저 병합되면 이 PR에는 P1-02 변경만 남습니다.